SCS1303

Introduction to Software Engineering

Section 4 · Requirements Engineering

Prof. K. P. Hewagamage

B.Sc. Computer Science · Year 1, Semester 1

University of Colombo School of Computing

Section 4 — Intended Learning Outcomes

By the end of this section, students should be able to:

LO1	Explain what a software requirement is and describe the three dimensions — functional, non-functional, and domain
LO2	Explain why ambiguous requirements are dangerous and identify quality properties of a good requirement
LO3	Describe the Requirements Engineering process and how it differs in plan-driven and agile contexts
LO4	Identify common elicitation techniques and explain how requirements are discovered from stakeholders
LO5	Explain what a Software Requirements Specification (SRS) is and how it compares to an agile product backlog
LO6	Describe how requirements are validated and managed throughout the project lifecycle
LO7	Describe how AI is beginning to assist in requirements engineering activities

Section 4 — Roadmap

Eight topics covering both plan-driven and agile requirements engineering

4.1

RE in both cultures — setting the scene

4.2

What is a requirement? Three dimensions

4.3

Requirement quality — ambiguity and precision

4.4

The RE process and feasibility study

4.5

Elicitation — how we discover requirements

4.6

Specification — SRS, backlog, and both cultures

4.7

Validation and requirements management

4.8

AI in Requirements Engineering

4.1

RE in Both Cultures

Connecting what you already know about Agile to Requirements Engineering

Why Requirements Matter

The most expensive mistakes in software happen here — at the very beginning

The most expensive bug in software

A defect found during requirements costs almost nothing to fix. The same defect found after deployment costs 100× more — because rework, redesign, retesting, and redeployment all compound.

Evidence from real projects:

~50%	of software defects can be traced back to requirements errors — not coding mistakes
~80%	of project rework is caused by poorly understood or changing requirements
Top cause	of project failure in Standish Group CHAOS reports: unclear or changing requirements

Getting requirements right is the single highest-leverage activity in software engineering.

Requirements in Both Cultures

The same discipline — very different in how it is practised

You already know that there are two main cultures in software development. They approach requirements very differently.

Plan-Driven Requirements

- All requirements defined and documented BEFORE building begins
- Written in a formal document called a Software Requirements Specification (SRS)
- Changes are controlled through a formal approval process
- Best when requirements are stable, or when the project is regulated or contractual

Agile Requirements

- Requirements are captured as user stories — short, simple, user-focused
- Stored in a product backlog that evolves throughout the project
- Detailed requirements emerge during sprint planning — just-in-time
- Best when requirements are uncertain or when customer needs evolve frequently

Both Cultures Are Important

Most software projects today use a mix.
You need to understand both approaches — and when to use which.

Aspect	Plan-Driven	Agile
How requirements look	Numbered sentences in a formal SRS document	User stories on a backlog: "As a…, I want…, so that…"
When are they written	Before development begins — all upfront	Just before they are needed – incrementally
How detailed	Complete, precise, formal, with measurements	Lightweight — enough to have a conversation
How change is handled	Formal change request — reviewed and approved	Reprioritise the backlog — quick and continuous
Main document	Software Requirements Specification (SRS)	Product Backlog with acceptance criteria

Section 4.2

What is a Requirement?

Defining the foundations — three dimensions every engineer must understand

What is a Requirement?

Definition

A requirement is a condition or capability that must be satisfied — needed by a user to achieve a goal, or required of a system to meet a standard or constraint.

A requirement can take three forms:

User Need

The goal

Expressed in the user's own language. What the person wants to achieve.

"I want to track my daily fitness."

System Rule

The capability

What the system must provide in order to meet the need.

"The system shall record daily step counts and display weekly charts."

Documented Form

The record

A written version of either of the above — the artifact that captures, communicates, and preserves the

A user story card: "As a user, I want to track my steps so that I can stay healthy."

Lifecycle of a Requirement

A requirement does not spring fully-formed from a meeting — it evolves

1	**Need identified**	A stakeholder expresses a goal, problem, or desire. Could be a user, manager, or regulator.
2	**Problem understood**	The gap between the current situation and the desired outcome is analysed. Why does this need exist?
3	**Requirement defined**	The need is formalised into a clear, actionable requirement — precise enough to design and test.
4	**Solution proposed**	A system feature or change is designed to fulfil the requirement. One requirement may have several design options.
5	**Conflicts managed**	One requirement may trigger others, or conflict with existing requirements. Conflicts must be resolved.
6	**Prioritised**	Not every requirement can be built. Decisions are made about what to include, defer, or drop.

From Need to Capability — and Why Ambiguity Kills

Misunderstood or vague requirements are a leading cause of project failure

User Need → **Requirement** → **System Capability**

A well-formed requirement

"The system shall allow a registered user to reset their password via a link sent to their registered email; the link shall expire after 24 hours."

The danger of ambiguity — one sentence, two systems

Vague: *"The system shall allow users to search for patients."*

Reading A: search by name across all clinics and all appointments (a large, complex feature)

Reading B: search by name within the currently selected clinic only (a small, simple feature)

The Three Dimensions of Requirements

Every requirement answers one of three questions: What? How well? Why?

WHAT — Functional

what the system does

- Features, behaviours, and services
- What the user can do with the system
- What the system must NOT do
- Business rules and workflow logic

HOW WELL — Non-Functional

how well the system does it

- Quality attributes — performance, security, reliability
- System-wide properties, not individual features
- Often drive major architecture decisions
- Must be measurable — not just 'fast' or 'reliable'

WHY — Domain

why the requirement exists

- Rules from the operating environment — laws, regulations
- Constraints that come from outside the system
- Often non-negotiable (e.g. legal requirements)
- Specific to an industry: healthcare, banking, aviation

These three dimensions interact — a domain rule (e.g. GDPR) can trigger a functional requirement (consent capture) and a non-functional one (data encryption).

Functional Requirements — The WHAT

What the system should DO for its users

Definition

Functional requirements describe the behaviours, services, and features of the system — what it does, what the user can do with it, and what it must never do.

Examples across different systems:

Online banking	*The system shall allow a registered user to transfer funds to another account.*
eCommerce	*The system shall apply a 10% discount when the basket total exceeds LKR 5,000.*
University portal	*The system shall generate and email a student's exam results within 2 hours of results being published.*
Fitness app	*The system shall record the user's daily step count and display a weekly bar chart.*

In Agile: functional requirements are captured as user stories.
In plan-driven: as numbered sentences in an SRS document.

Non-Functional Requirements — The HOW WELL

How well the system performs — quality attributes that matter just as much as features

Definition

Non-functional requirements describe the quality attributes of the system — how reliably it runs, how fast it responds, how secure it is, how easy it is to use. They are system-wide, not tied to individual features.

Six common categories:

Performance

The system shall process a search query and return results in under 1 second for 95% of requests.

Reliability

The system shall be available 99.9% of the time — no more than 8 hours of downtime per year.

Security

Only registered administrators shall be able to modify student grade records.

Usability

A new user shall be able to complete account registration in under 3 minutes with no training.

Maintainability

A developer shall be able to add a new payment method without modifying existing payment code.

Interoperability

The system shall accept patient data formatted using HL7 FHIR standards.

Non-Functional Requirements Must Be Measurable

A vague NFR is not a requirement — it is a wish

The most common mistake with non-functional requirements

Bad (vague)	Good (measurable)
✗ The system shall be fast.	✓ The system shall respond to any search query in under 1 second for 95% of requests under normal load.
✗ The system shall be user-friendly.	✓ A first-time user shall complete the registration form in under 3 minutes without requesting help.
✗ The system shall be highly available.	✓ The system shall be available 99.9% of the time, measured over any 30-day period.
✗ The system shall be secure.	✓ All patient data shall be encrypted using AES-256 both at rest and in transit.

If you cannot write a test for it, it is not a requirement. NFRs must be precise enough to verify.

Domain Requirements — The WHY

Rules that come from outside the system — and are usually non-negotiable

Definition

Domain requirements are constraints that arise from the industry, legal regulations, or operating environment. They explain WHY certain behaviours are necessary — beyond what any user personally wants.

How domain requirements work — they trigger functional and non-functional requirements:

Healthcare

Patient records must be retained for 10 years (regulatory requirement)

FR: System shall archive patient records with a 10-year retention policy

NFR: Storage must maintain 99.9% data integrity over the retention period

Banking

All transactions must comply with Anti-Money Laundering (AML) laws

FR: System shall flag transactions above LKR 1 million for compliance review

NFR: Audit logs must be immutable and retained for 5 years

Education

Grades must follow the government-defined grading scale

FR: System shall calculate GPA using the national 4.0 grading standard only

NFR: GPA calculations must be independently auditable by the Ministry

Three Dimensions Together — A Real Example

Same system, three kinds of requirement — a hospital appointment app

For any system, you will always find requirements from all three dimensions working together:

Functional

- The system shall let a patient book an appointment with an available doctor.
- The system shall send a reminder SMS 24 hours before an appointment.

Non-Functional

- The booking page shall load in under 2 seconds for 95% of requests.
- The system shall be available 99.9% of the time — including weekends and public holidays.

Domain

- Patient medical records shall be retained for 7 years (Health Ministry regulation, Sri Lanka).
- Prescriptions shall only be issued by practitioners registered with the SLMC.

The Three Dimensions in Practice — Examples

Same system (a hospital appointment app), three kinds of requirement

Dimension	Example Requirement	How you recognise it
Functional	"The system shall let a patient book an appointment with an available doctor."	Describes a behaviour or service the user invokes
Functional	"The system shall send a reminder 24 hours before an appointment."	A discrete, testable system action
Non-Functional	"The booking page shall load in under 2 seconds for 95% of requests."	A measurable quality — performance
Non-Functional	"The system shall be available 99.9% of the time."	System-wide quality — availability
Domain	"Patient records shall be retained for 7 years (health regulation)."	Comes from law / industry — non-negotiable
Domain	"Prescriptions shall only be issued by licensed practitioners."	Rule of the operating environment

Note: a vague non-functional requirement ("the system shall be fast") is worthless. NFRs must be quantified

Section 4.3

Requirement Quality

Ambiguity, precision, and the properties of a good requirement

The Danger of Ambiguity

One sentence — two completely different systems

Ambiguous requirement:
"The system shall allow users to search for patients."

Developer A reads it...

Search by name within the currently selected clinic only — a small, simple feature taking 2 days

Developer B reads it...

Search by name across ALL clinics and ALL appointments — a large, complex feature taking 3 weeks

Result: two developers build two different systems — neither of which is what the customer wanted. Ambiguity is the enemy.

Removing Ambiguity — Before and After

Every word in a requirement should have exactly one meaning to every reader

Step by step — sharpening one requirement:

Version 1 (original)

"The system shall allow users to search for patients."

Who is searching? By what? In which clinic? One appointment or all? Completely ambiguous.

Version 2 (better)

"The system shall allow clinic staff to search for a patient by full name within the currently selected clinic."

Better — now we know who, what field, and the scope. But still missing: what if multiple patients share the same name?

Version 3 (good)

"The system shall allow clinic staff to search for a patient by full name within the currently selected clinic. If multiple patients share the same name, results shall display the patient's date of birth and clinic ID to distinguish them."

Now it is clear, complete, and testable. A developer can build it; a tester can verify it.

Properties of a Good Requirement

A requirement should satisfy all of these — if any is missing, it needs more work

Clear

Only one reasonable interpretation is possible. Every reader — engineer, tester, customer — understands it identically.

Complete

Nothing essential is missing. The requirement says everything needed to design and test the feature.

Consistent

Does not contradict any other requirement in the document. No two requirements can conflict.

Verifiable

You can write a test for it. If you cannot think of how to test it, the requirement is too vague.

Traceable

Linked back to a stakeholder need and forward to a design decision or test case. You know WHY it exists.

Feasible

Technically possible to implement within the available budget, time, and technology.

Section 4.4

Requirements Engineering

The systematic discipline — process, feasibility, and two stages

What is Requirements Engineering?

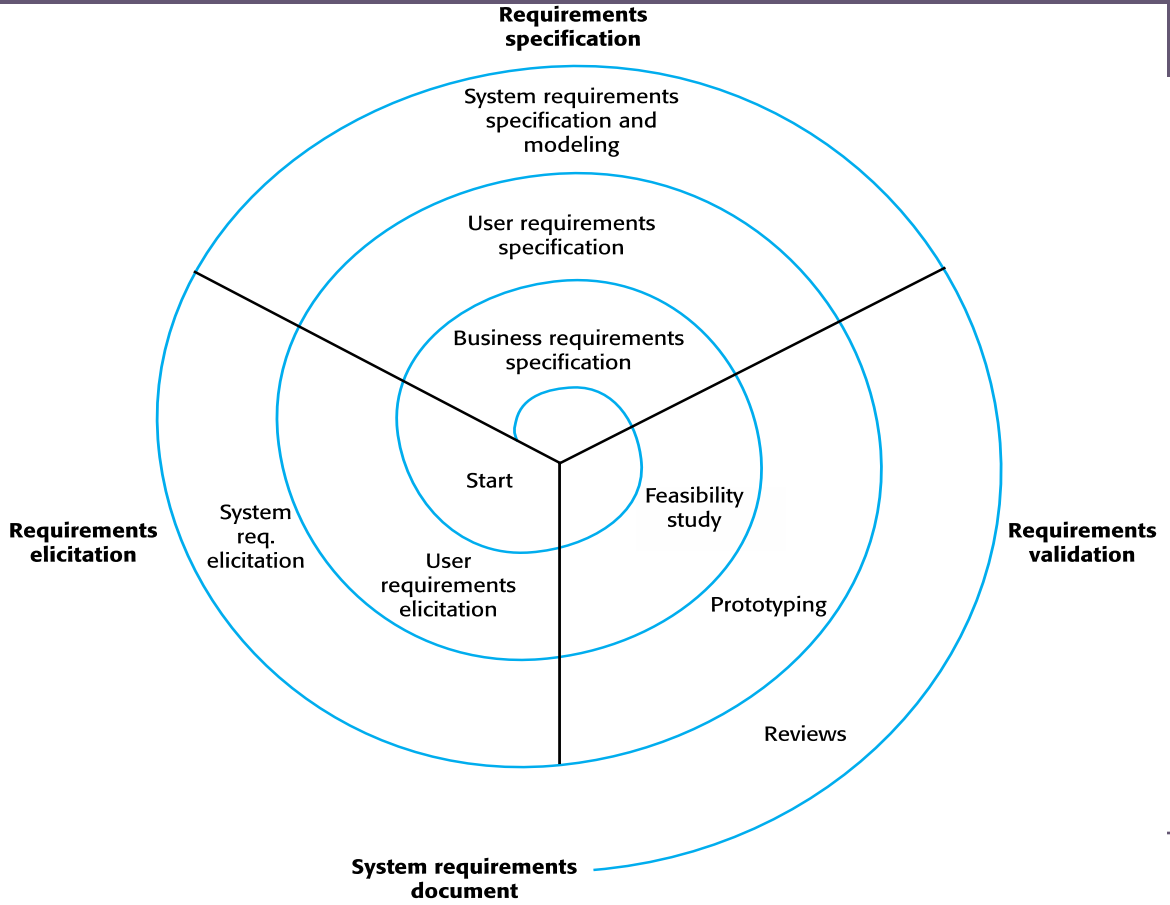
Definition

Requirements Engineering (RE) is the systematic and disciplined process of discovering, analysing, documenting, validating, and managing the requirements for a software system. It covers WHAT the system must do, HOW WELL it must do it, and WHY certain behaviours are needed.

Why call it 'engineering'?

Systematic process	Follows clear, repeatable steps — not ad hoc collection in a single meeting
Rigorous	Avoids ambiguity, ensures traceability, and verifies completeness
Problem-solving	Resolves conflicts between stakeholders, works within real-world constraints
Tool and method use	Uses models, templates, diagrams, and formal notations where needed
Iterative by nature	Adapts as the system and stakeholder understanding evolves over time

A spiral view of the requirements engineering process



The Requirements Engineering Process

Five activities that form a cycle — not a one-time checklist

RE is not a straight line from start to finish. It is an iterative cycle — each activity can lead back to an earlier one as understanding deepens.

↑ *RE is iterative — discovering new information in later stages often requires returning to earlier stages*

Two streams run throughout the entire project:

Requirements Development

Discover, analyse, specify, and validate what is needed — produces the agreed set of requirements

Requirements Management

Handle changes, maintain traceability, keep everything consistent as the project evolves over time

Feasibility Study

Before spending months on requirements — is this project worth doing at all?

What is a feasibility study?

A short, focused assessment carried out at the very beginning to determine whether the proposed software system is realistic, financially justified, and aligned with the organisation's goals.

Five types of feasibility — each asks a different question:

Technical:	Can it be built with available technology and skills? Are there technical risks that could stop the project?
Economic:	Do the benefits justify the cost? Is there a clear return on investment? Is the budget sufficient?
Operational:	Will the organisation actually use it? Does it fit how people work? Will users accept and adopt it?
Legal:	Does it comply with relevant laws, data protection regulations, licences, and contractual obligations?
Schedule:	Can it be built within the required timeframe? Are there external deadlines that must be met?

Output of a feasibility study: a recommendation to proceed, modify scope, or abandon the project — before any significant investment is made.

Common Types of Feasibility

Type	Description
Technical	Assess if current technology supports the solution
Economic	Analyze costs vs. expected benefits (cost-based / time-based)
Legal	Compliance with laws, licenses, data protection, etc.
Operational	Can users, processes, and environment support the solution?
Schedule	Can the system be built within required timeframes?
Resource	Are personnel, infrastructure, and tools available?
Cultural	Will the system align with user and organizational culture?
Market/Real Estate	(For physical systems) Is the market location viable?
Financial	Will the funding and financial flows sustain development?

The Two Stages of RE

Development produces requirements; Management keeps them under control

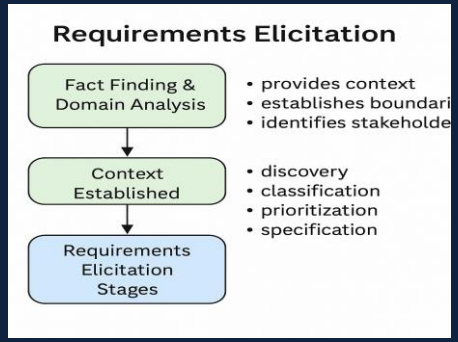
1. Requirements Development

- Goal: discover, analyse, document, and validate needs
- Elicitation — interviews, workshops, observation, discovery
- Analysis — resolve conflicts, prioritise, model
- Specification — write clear, testable requirements
- Validation — check correctness, completeness, consistency
- → Output: an agreed, baselined set of requirements

2. Requirements Management

- Goal: control change and maintain integrity over time
- Change control — assess and approve changes systematically
- Traceability — link requirements to design, code, tests
- Version control — track requirement evolution
- Impact analysis — understand the cost of each change
- → Runs continuously through development and maintenance

Section 4.5



Requirements Elicitation

Discovering what stakeholders really need — not just what they say they want

Getting from "we want a system" to grounded, validated needs — classic fact-finding and domain analysis, plus the modern discovery techniques that have transformed practice.

What is Requirements Elicitation?

An important distinction

Elicitation is NOT simply 'requirements gathering' — requirements are rarely lying around ready to be collected. They must be actively drawn out, discovered, and often co-created through conversation with stakeholders.

Why is it difficult?

- Stakeholders often do not know exactly what they want until they see a working version
- Different stakeholders have different — and sometimes conflicting — needs and priorities
- People describe imagined solutions rather than real underlying needs when asked "what do you want?"
- Organisational politics and personal agendas can influence or distort what people say
- Requirements evolve as the project progresses and understanding deepens

The golden rule: don't ask "What do you want?" — ask "What do you DO?"
People reveal real needs through their actual work.

Who Are Stakeholders?

Anyone who has an interest in the system — or is affected by it

Definition

A stakeholder is any person, group, or organisation that has an interest in the system being developed — as a user, funder, regulator, or someone whose work the system affects.

Common stakeholder groups:

End users — The people who will use the system daily — their needs must drive the requirements

Managers/clients — Fund the project and define business goals — often set priority and scope

Domain experts — Know the business or industry deeply — hospital staff, accountants, legal experts

Technical staff — Developers, DBAs, architects — understand constraints and feasibility

Regulators — Government agencies, standards bodies — set the rules the system must comply with

Operations and support — Maintain the system after delivery — their needs affect maintainability requirements

Fact-Finding and Domain Analysis

Understand the world before you try to change it

Fact-Finding

- "Elicitation without fact-finding is exploring without a map"
- Systematically collect information about the organisation, stakeholders, and existing systems
- Identify stakeholders across all levels
- Study current workflows, tasks, and procedures
- Assess constraints — cost, technical, policy
- Outputs: problem context, system scope, business objectives, early constraints & risks

Domain Analysis

- Study the problem domain — its concepts, rules, processes, and vocabulary
- Gain deep knowledge of how the business actually works
- Identify domain-specific constraints (laws, standards, best practice)
- Discover reusable knowledge from similar systems
- Bridge the knowledge gap between stakeholders and engineers
- Speeds up elicitation and helps anticipate future needs

Agreeing scope up front is the single most effective way to reduce later requirement instability — but in continuous contexts, scope itself is revisited each cycle.

Activities in Fact Finding

Key Activities:

- Identify stakeholders and key users across levels
- Interview analysts with domain expertise
- Analyze existing systems (if any)
- Study current workflows, tasks, and procedures
- Investigate domain models and technologies
- Assess system constraints (cost, technical, policy)
- Conduct site visits, document reviews, and observations

Outputs of Fact Finding

Deliverables

- A clear **problem context** description/statement
- Defined **system scope** (boundaries and interfaces)
- List of **business objectives** and goals
- Overview of **existing processes** and stakeholders
- Preliminary identification of system constraints and risks

If everyone involved agrees upfront to the scope of the system, the instability of requirements can be minimized

Domain Analysis in Requirements Engineering

Domain analysis is the process of **studying the problem domain** to understand its concepts, rules, processes, and constraints.

It helps software engineers gain deep knowledge of **how the business works**, which is essential for deriving valid and useful requirements.

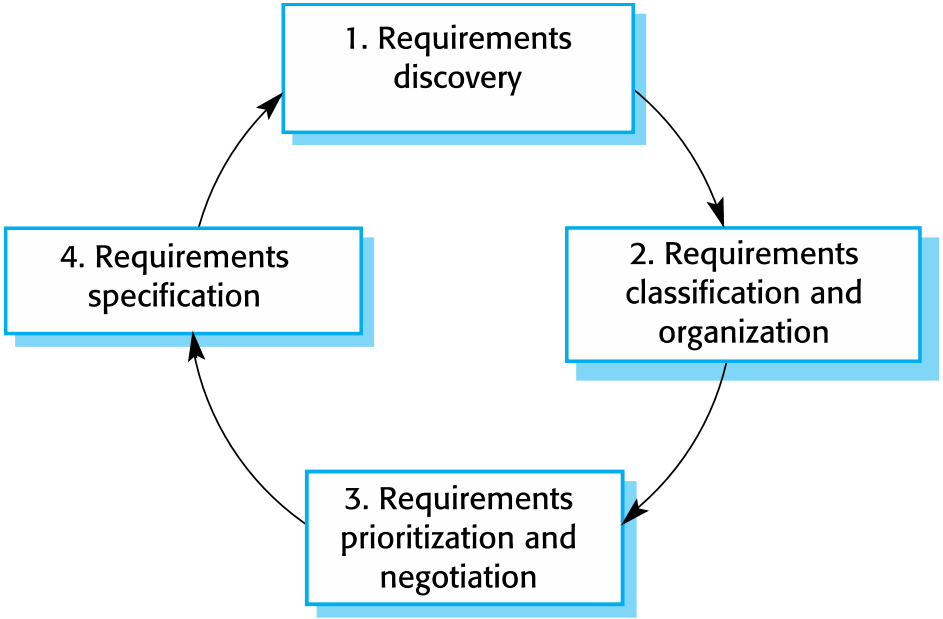
Objectives of Domain Analysis

- Understand the **environment** in which the system will operate
- Identify **domain concepts**, terms, and relationships
- Recognize **domain-specific constraints** (e.g., laws, standards, best practices)
- Discover **reusable knowledge** from similar systems or previous projects
- Bridge the knowledge gap between stakeholders and developers

Benefits of Domain Analysis

- Speeds up requirement elicitation
- Helps in **anticipating future needs and changes**
- Supports the development of a **better, more aligned system**
- Reduces miscommunication between stakeholders and engineers
- Encourages reuse of domain models and components

The Stages of Requirements Elicitation – Activities



Elicitation — Four Key Activities

The elicitation process is structured around four core activities

1	**Discovery**	Engaging with stakeholders to identify needs. Asking questions, listening carefully, observing actual work practices. This is where most of the learning happens.
2	**Classification & Organisation**	Grouping related requirements together. Organising them into functional, non-functional, and domain categories. Structuring for clarity.
3	**Prioritisation & Negotiation**	Not all requirements can be built — especially not at once. Ranking requirements by importance and resolving conflicts between competing stakeholder needs.
4	**Specification**	Documenting the elicited requirements in a clear, structured form. Writing user stories, SRS sections, or use cases that can be reviewed, tested, and traced.

Elicitation Techniques

Different situations call for different approaches to discovering requirements

Interviews

Best for: Gaining rich, qualitative insight from individual stakeholders

One-on-one conversation — ask open questions, listen, probe deeper. Ask 'why' repeatedly to uncover the real need beneath the stated request.

Workshops

Best for: Bringing multiple stakeholders together to agree shared requirements

Facilitated group session. Useful for resolving conflicts between stakeholder groups and reaching consensus on scope and priority quickly.

Observation

Best for: Understanding what people actually do — not just what they say they do

Watch users work in their real environment. People routinely do things differently from how they describe them. Reveals tacit knowledge.

Prototyping

Best for: When requirements are vague — show something to get feedback

Build a quick mock-up or sketch of the system. Let stakeholders react to what they see. Reveals requirements that are impossible to articulate verbally.

Document analysis

Best for: Understanding regulations, existing systems, and domain rules

Review existing documentation: current system manuals, regulations, reports, contracts. Essential for identifying domain requirements.

Surveys / questionnaires

Best for: Gathering input from many stakeholders quickly

Structured questions distributed widely. Good for quantitative data or reaching stakeholders who cannot attend workshops.

Discovery Techniques — Classic Toolkit

Match the technique to the stakeholder and the information you need

Category	Techniques	Best for
Conversational	Interviews, workshops, focus groups, brainstorming	Rich, qualitative understanding; exploring the unknown
Observational	Direct observation, shadowing, ethnography, "day in the life"	Uncovering tacit knowledge & what people actually do (vs. say)
Analytical	Document analysis, system archaeology, competitor analysis	Existing systems, regulations, prior art
Synthetic	Prototypes, scenarios, personas, questionnaires/surveys	Validating ideas; reaching many stakeholders at scale

The golden rule of discovery

Don't ask "What do you want?" — ask "What do you do?" People describe imagined solutions when asked what they want, but reveal real needs, workflows, and frustrations when asked about their actual work. Use open-ended prompts, involve diverse stakeholders, watch for bias and preconceived solutions, and record "**To Be Determined (TBD)**" items for follow-up.

Defining the Problem and the Scope

A sharp problem statement prevents a thousand arguments later

Writing a good Problem Statement

- State the problem, not a solution
- Be specific — avoid vague language
- Name key stakeholders and their pain points
- Include measurable impact or expected benefit
- Align with business goals or customer objectives
- Keep it short and focused on the core issue

Defining Scope (in & out)

- List everything you might imagine the system doing
- Explicitly EXCLUDE what is out of scope (this is the hard part)
- Narrow if too broad; clarify high-level goals if too narrow
- Define boundaries and external interfaces
- Agreeing scope up front minimises requirement instability
- In Agile: scope is a negotiable, evolving boundary

Example: a university registration system — is course-catalogue management in scope? Is fee payment? Is timetabling? Each answer reshapes the project.

Identifying the Problem and Establishing the Scope

Identifying the Problem

- A problem can be expressed as:
  - A **difficulty** faced by users or customers in achieving their goals, or
  - An **opportunity** to enhance current operations (e.g., improve productivity, reduce cost).
- The solution often involves the development of a new or modified software system.

Writing an Effective Problem Statement

- Be specific and avoid vague language.
- Identify **key stakeholders** and their pain points.
- Include **measurable impacts** or expected benefits.
- Align with **business goals** or customer objectives.

Defining the Scope

- Clearly define **what is included** and **what is excluded** from the system.
- A good problem statement:
  - Is **short and focused**
  - Captures the **core issue or opportunity**
  - Helps prevent **requirement instability** during later stages.

Common Challenges in Elicitation

Every requirements engineer faces these — knowing them helps you prepare

Stakeholders cannot articulate their needs

People know what frustrates them but struggle to describe what they actually want. Techniques: show examples, use prototypes, ask 'walk me through a typical day'.

Different stakeholders want different things

The marketing team wants feature X; the finance team opposes it because of cost; the legal team says it may violate regulations. Conflict is normal — it must be managed.

Requirements keep changing

New stakeholders join; the business environment changes; technology evolves. This is why RE is iterative — not a single event at the project start.

Stakeholders describe solutions, not needs

"I want a button that does X" is a design decision, not a requirement. The engineer's job is to ask 'why?' until the underlying need is uncovered.

Implicit requirements

Things everyone assumes are obvious — but have not been written down. Often discovered late. Example: users assume the system will work on mobile, but nobody said so.

PART 5

Analysis & Modelling

Refining raw requirements into structured, analysed models — detecting conflicts and gaps, and connecting requirements to design through UML, DFDs, and Domain-Driven Design.

Requirements Analysis & Modelling

Why model? To find what words alone hide

What analysis achieves

- Detect inconsistencies, omissions, ambiguities, conflicts
- Validate requirements against goals and constraints
- Establish clear relationships among requirements
- Reveal gaps that elicitation missed
- Provide the foundation for design, specification & validation
- Turn a pile of statements into a coherent system view

Modelling techniques

- Use Case Diagrams — functional context & actors
- Activity Diagrams — workflows and process logic
- Class / ER Diagrams — structure and data
- Data Flow Diagrams (DFD) — how data moves & transforms
- System Context Diagram — boundaries & external interfaces
- State Machines — behaviour over time for reactive systems

Models are communication tools, not bureaucracy:
a single diagram can surface a contradiction that ten pages of prose would hide.

Requirements Analysis & Modeling

To refine, organize, and analyze the elicited requirements to ensure clarity, consistency, completeness, and relevance before formal specification.

- Detect inconsistencies, omissions, ambiguities, and conflicting requirements.
- Validate that requirements align with stakeholder goals and system constraints.
- Establish clear relationships among requirements.
- Serve as a foundation for:
  - System Design
  - System Validation
  - Requirement Specification

Modeling in Requirements Analysis

- Helps visualize and structure complex requirements.
- Reveals gaps in elicitation.
- Assists in stakeholder communication and validation.

Common Modeling Techniques:

- **UML Models**:
  - Use Case Diagrams (functional context)
  - Class Diagrams (structure)
  - Activity Diagrams (workflow)
- **Data Flow Diagrams (DFD)** – Data processing logic
- **Entity Relationship Diagrams (ERD)** – Data structure logic
- **System Context Diagram** – System boundaries and external interactions

Requirements Classification

Classify requirements into meaningful categories to improve understanding and management:

- **Functional Requirements** – Describe system behavior and functionalities
- **Non-Functional Requirements** – Define system quality attributes (e.g., performance, security)
- **Design Constraints** – Technology, standards, or regulatory limits
- **Environmental Constraints** – Physical, hardware, or network conditions

Note: Requirements classification often overlaps with evaluation and prioritization.

Requirements Prototyping (Optional Support Tool)

Purpose:

- Clarify unclear requirements
- Gain early user feedback
- Reduce risk of misunderstanding

b. Types of Prototypes:

- **Throwaway Prototypes** – Used for clarification and discarded after
- **Evolutionary Prototypes** – Gradually evolved into final system

c. Best Practices:

- Clearly define scope: what is included and what is not
- Use scripts to guide evaluation
- Collect and document user feedback systematically

d. When to Avoid:

- When rapid delivery is prioritized over clarity
- When users confuse prototype with final system

Output of Analysis and Modeling

- Validated and classified requirements
- Conceptual models representing the system scope and requirements
- Prototype feedback (if used)
- Identified gaps for further elicitation
- Ready-to-refine input for Requirement Specification phase

Prioritisation

You can never build everything.

Deciding what to build first —

MoSCoW, the Kano model, and Weighted

Shortest Job First.

Why Prioritise — and on What Basis?

Limited time, budget, and capacity force hard choices

Prioritisation helps you

- Decide what to implement first
- Distinguish critical from optional features
- Manage stakeholder expectations honestly
- Plan release cycles around value
- Reduce risk by sequencing the riskiest items early
- Avoid the trap of treating everything as "high priority"

Criteria to weigh

- Benefit to the user — does it solve a real pain?
- Cost & effort to develop
- Risk & uncertainty
- Dependencies — can it stand alone?
- Ease of deployment & adoption
- Strategic / regulatory importance

The hardest truth: if everything is a priority, nothing is. Forcing a ranking is itself the value — it makes trade-offs explicit and accountable.

Prioritising Requirements — MoSCoW

Not everything can be built — deciding what matters most

Why prioritise?

Budget, time, and team capacity are always limited. Prioritisation forces a clear decision about what must be built, what is desirable, and what can wait — preventing scope creep.

The MoSCoW technique — a widely used, simple prioritisation framework:

M — Must have

Essential. Without this, the system will fail or be completely unusable. Non-negotiable.

A hospital booking system MUST allow patients to book appointments.

S — Should have

Important but not vital for first release. Can be temporarily worked around if

SHOULD send confirmation emails — useful but the system works without it.

C — Could have

Desirable. A nice-to-have feature that adds value but is not critical to core function.

COULD show appointment history. Useful, but patients can survive without it.

W — Won't have

Explicitly agreed NOT to build in this release. Records the decision — not a permanent rejection.

WON'T integrate with health wearables — deferred to a future version.

Section 4.6

Requirements Specification

Documenting requirements — the SRS, the backlog, and both cultures

What is Requirements Specification?

Definition

Requirements specification is the activity of documenting the discovered and analysed requirements in a clear, structured form that all stakeholders can understand and that developers, testers, and designers can use to do their work.

Three levels of formality — choose based on the project context:

Natural language	Plain sentences — accessible to everyone but can be ambiguous. Most common for user-level requirements. *"The system shall allow registered users to reset their password via email."*
Structured language	Templates and fixed formats that reduce ambiguity while staying readable. Good for system requirements. *ID: UC-004 | Actor: Registered user | Trigger: User clicks 'Forgot password' | Post-condition: Reset email sent*
Graphical models	Diagrams (use cases, activity diagrams) that show structure and flow visually. Excellent for communication. *A use case diagram showing 'Reset Password' with the Registered User actor and the Email System actor*

Writing One Requirement Well

Before any document — the craft is stating a single requirement precisely

Every good requirement goes through refinement — from a rough user need to a precise, testable system statement:

User need (rough)

"As a patient, I want to reset my forgotten password so that I can log in again."

Clear intent — but who can do it? How? Any limits? All of these are unstated and need answering.

▼

System requirement

"The system shall allow a registered patient to request a password reset and shall email a reset link to the patient's registered address."

Now precise about who (registered patient), what (reset link), and how (email). But missing measurable limits.

▼

Refined and testable

"… The reset link shall expire 24 hours after issue and shall be usable only once."

Now it is testable. A developer knows exactly what to build. A tester can verify it. This is a good requirement.

The Software Requirements Specification (SRS)

The formal document used in plan-driven requirements engineering

What is an SRS?

An SRS is a complete description of the software system to be built — covering what it must do (functional), how well it must do it (non-functional), and the constraints it must operate under. It is the primary reference document for the whole project.

A good SRS is:

Clear	Only one valid interpretation of every requirement — ambiguity is removed before writing
Complete	All functional, non-functional, and domain requirements are present — nothing assumed
Consistent	No requirement contradicts another. All terms are used the same way throughout
Verifiable	Every requirement can be tested. If no test can be written, the requirement must be rewritten
Traceable	Each requirement links back to a stakeholder need and forward to a design or test case

When to use an SRS: safety-critical systems · regulated industries · fixed-price contracts · large distributed teams who need a shared written reference

SRS — Typical Structure

Based on the IEEE standard for requirements documentation

An SRS is organised into sections so that different readers can find what they need quickly:

1	**Introduction**	Purpose and scope of the system, intended readers, definitions and abbreviations, references, document overview
2	**Overall Description**	System context, the product's place in the wider environment, user characteristics, assumptions and constraints
3	**User Requirements**	What users need, expressed in plain language or use cases. Readable by non-technical stakeholders including the client
4	**System Requirements**	Detailed functional and non-functional requirements. Written precisely for developers, architects, and testers
5	**Models and Diagrams**	Use case diagrams, data flow diagrams, entity-relationship diagrams, and other visual models of the system
6	**Appendices**	Glossary of terms, references, version history, any supporting information not in the main body

The Product Backlog — Agile Specification

In Agile, the backlog IS the living requirements document

What is a product backlog?

The Product Backlog is an ordered, ever-evolving list of everything the product might need. It is never 'finished' — it grows and changes as understanding deepens. The top items are detailed and ready to build; items further down are rough and vague.

How requirements get more detailed as they get closer to being built:

Epic — *"Appointment booking" — covers everything to do with booking, cancelling, and rescheduling.*

◀ A large body of work — too big to build in one sprint. A rough description of a major feature area.

Feature — *"Allow patients to book new appointments with available doctors"*

◀ A meaningful capability within an epic. Still broad but more concrete.

User Story — *"As a patient, I want to search for available appointment slots by date so that I can pick a convenient time."*

◀ A small, buildable piece of functionality completable in days.

Acceptance Criteria — • *At least 3 available slots shown · Sorted by time · Slots update in real time*

◀ Specific conditions that must be true for the story to be 'done'.

SRS vs Product Backlog — Both Cultures Side by Side

Same purpose: define what to build. Very different approaches.

	SRS (Plan-Driven)	Product Backlog (Agile)
When written	All upfront before development	Incrementally — just before needed
Who reads it	All stakeholders, esp. developers and testers	Development team and Product Owner
How detailed	Complete and precise — formal document	Top items detailed; lower items rough
How it changes	Formal change control — reviewed approval	Continuous reprioritisation — flexible
Document form	Structured chapters, numbered requirements	Ordered list of stories with acceptance criteria
Best for	Regulated, contractual, safety-critical	Evolving requirements, fast feedback loops

Neither approach is universally better — the right choice depends on the project context.
Many organisations use both on different parts of the same project.

Common Problems in Requirements Documents

Knowing these problems helps you avoid them when writing requirements

Mixing design and requirements

Writing HOW the system will be built instead of WHAT it must do. A requirement describes behaviour — a design describes structure.

✗ "The system shall store passwords using bcrypt hashing."

✓ "The system shall protect user passwords against unauthorised access."

Vague, untestable language

Words like 'fast', 'user-friendly', 'robust', 'flexible' mean different things to different people and cannot be tested.

✗ "The system shall be user-friendly."

✓ "A new user shall complete registration in under 3 minutes."

Missing requirements

Nobody documented something 'everyone knows' — it gets discovered when users test the delivered system and find it absent.

✗ Deployment of an app with no mention of mobile browser support.

✓ "The system shall function correctly on Chrome, Safari, and Firefox on mobile devices."

Inconsistency

One section says something; another section says the opposite. Often caused by different people writing different parts.

✗ Section 3: 'All users must log in.' Section 5: 'Guest users may access the catalogue.'

✓ One term, one definition, used consistently throughout.

Section 4.7

Validation and Management

Checking correctness · controlling change · maintaining traceability

Requirements Validation

Are these the right requirements? Are they good enough?

Verification

"Did we build the product right?"

Checking that the system matches its specification. Formal methods: code reviews, static analysis.

Validation

"Did we build the right product?"

Checking that the requirements reflect what the stakeholders actually need. Reviews, prototypes, acceptance tests.

The 5C Validation Check — five questions to ask about every set of requirements:

Correct: Do the requirements reflect the stakeholders' real needs — not misunderstandings?

Complete: Are all necessary requirements present? Have any important ones been left out?

Consistent: Are there any contradictions between requirements?

Capable: Are the requirements technically and financially feasible to implement?

Checkable: Can each requirement be verified or tested? If not, it needs to be rewritten.

Validation Techniques

How we actually check that requirements are correct before development begins

Requirements reviews	A structured meeting where stakeholders read through requirements together, looking for errors, inconsistencies, and gaps. Can be formal (with a chair and recorder) or informal.
Prototyping	Build a quick, rough version of the system interface and let real users react to it. Reveals requirements that cannot be articulated in words but become obvious when seen.
Test-case generation	For each requirement, try to write a test case. If you cannot write a clear test, the requirement is ambiguous or incomplete and must be rewritten before it is accepted.
Acceptance criteria	In agile contexts, requirements are validated through concrete acceptance criteria written before development begins — if the feature meets all criteria, the requirement is satisfied.
Walkthroughs	The author presents the requirements step-by-step to a small group who ask questions and identify issues. Less formal than a review — good for early-stage requirements.

Why Requirements Change

Change is not failure — it is the normal condition of software projects

The reality

Requirements almost always change. Business needs evolve. Stakeholders revise their priorities. Technology shifts. Understanding deepens. A good process manages change — it does not pretend change will not happen.

Four common reasons requirements change:

Evolving business needs	Business goals, regulations, and market conditions change over time — so must the system that supports them.
Stakeholder diversity	Different users and funders often discover they want different things as they see the system taking shape.
Technology shifts	New platforms, APIs, or integration requirements emerge after initial planning that were not anticipated.
Understanding deepens	What the team needed at the start often differs from what they discover they need after working with the system.

"Walking on water and developing software from a frozen specification are equally easy — if both are frozen." — Ed Yourdon

Requirements Management — Two Pillars

Managing requirements throughout the project lifecycle

What is requirements management?

Requirements management is the ongoing process of tracking requirements, controlling changes to them, and ensuring that every requirement remains linked to the work being done. It runs from the first sprint to final delivery.

Pillar 1: Change Management

- Establish a baseline — an agreed, approved set of requirements
- When a change is proposed: evaluate the impact, cost, and risk before deciding
- Only implement approved changes — no informal, uncontrolled changes
- In Agile: the backlog IS the living baseline — reprioritisation is continuous

Pillar 2: Requirements Traceability

- Every requirement is linked to the stakeholder need that drove it
- Every requirement links forward to a design decision and a test case
- Enables impact analysis: 'if this changes, what else breaks?'
- Helps detect missing features and features with no requirement (scope creep)

The Change Management Process

A structured path from request to approved and implemented change

When someone proposes a change to an agreed requirement, the following process applies:

1	**Submit change request**	The person requesting the change documents it: what to change, why, urgency, and the expected impact.
2	**Analyse the impact**	The team evaluates: which other requirements are affected? How much will it cost? What is the schedule impact?
3	**Make the decision**	A decision is made: Approve · Reject · Partially approve · Defer to a later release. This is documented.
4	**Implement if approved**	The approved change is made to all affected documents, designs, and tests. Not just the SRS — everything.
5	**Verify and close**	The change is verified as correct. All traceability links are updated. The change request is closed.

Requirements Traceability

Following a requirement through its entire lifetime — forward and backward

What is traceability?

Traceability is the ability to follow a requirement's life in both directions — from the original stakeholder need that created it, through to every design decision, line of code, and test case that implements and verifies it.

Two directions of traceability:

Backward traceability

From each requirement back to the stakeholder need or business objective that drove it. Answers: Why does this requirement exist? Who asked for it?

Forward traceability

From each requirement forward to the design elements, code components, and test cases that implement and verify it. Answers: Is this requirement built? Is it tested?

Why it matters:

• If a requirement changes — impact analysis tells you exactly what else must change · • Detect scope creep: features with no linked requirement are unauthorised additions · • Regulatory compliance: auditors can verify every legal requirement is implemented and tested

Section 4.8

AI in Requirements Engineering

How AI tools are beginning to change the way requirements work is done

AI is Changing Requirements Engineering

A forward look — the tools and practices entering the profession

Why RE is a natural fit for AI

Requirements engineering is mostly language work — reading interviews, writing documents, detecting ambiguity, and reformatting between specifications. These are exactly the tasks that modern AI tools handle well.

Where AI helps — and where humans remain essential:

AI does well...

- ✓ Detecting ambiguous or vague language
- ✓ Drafting user stories from rough notes
- ✓ Suggesting acceptance criteria for a story
- ✓ Summarising long interview transcripts
- ✓ Checking requirements against a standard format

Humans remain essential...

- ★ Deciding which stakeholder's need takes priority
- ★ Negotiating conflicts between stakeholders
- ★ Ethical and legal judgement on what is acceptable
- ★ Building trust and reading the room with stakeholders
- ★ Accountability — if a requirement fails, the engineer owns it

AI in Elicitation and Specification

The two RE stages where AI saves the most time today

AI in Elicitation and Discovery

- Transcribe and summarise stakeholder interviews and workshop recordings automatically
- Identify themes and group related needs from large sets of user feedback
- Suggest follow-up questions the interviewer might have missed
- Detect vague or contradictory statements in rough interview notes

AI in Specification

- Draft user stories and acceptance criteria from a feature description
- Rewrite a vague requirement into a clear, testable system requirement
- Generate SRS sections from a set of rough notes and requirements

AI drafts — a human reviews, decides, and owns. The engineer is responsible for the requirements, not the tool.

Using AI in RE — Important Cautions

AI makes requirements engineering faster — it does not make it easier to get right

These risks apply whenever AI is used for any requirements engineering task:

⚠ AI can invent requirements

AI may generate plausible-sounding requirements that no stakeholder ever expressed. Always trace every requirement back to a real person or a real source document.

⚠ Fluent output feels authoritative

Well-written AI output looks convincing. Teams sometimes accept it without the scrutiny they would apply to a colleague's work. Stay critical — read it carefully.

⚠ AI cannot validate its own output

If AI both writes a requirement AND checks it, it simply confirms its own assumptions. Always have an independent human review AI-generated requirements.

⚠ Confidentiality risk

Requirements often contain sensitive business and personal information. Check the tool's data-handling policies before pasting anything into a public AI tool.

⚠ The engineer is still responsible

If an AI-suggested requirement causes the system to fail, the engineer who accepted it is accountable. Professional responsibility does not transfer to the tool.

Common Pitfalls — What Goes Wrong in Requirements Engineering

Knowing these helps you avoid them in your own projects:

⚠ **Solutioning too early**	Capturing a stakeholder's proposed solution instead of their underlying need. Always ask 'why?' to get back to the real problem.
⚠ **Ambiguity left unresolved**	Vague terms (fast, user-friendly) that mean different things to different people — discovered only at acceptance time when it is expensive.
⚠ **Missing non-functional requirements**	Specifying what the system does but not how well. Then discovering at launch that it is too slow, insecure, or unusable under real load.
⚠ **Forgetting some stakeholders**	Building what one group wants while ignoring needs of other affected groups — discovered when the system is delivered and rejected by users.
⚠ **The document becomes the goal**	Optimising for a complete, polished SRS rather than genuine shared understanding. The artifact is a means — not the end in itself.
⚠ **No change management**	No baseline, no traceability, no impact analysis — every change becomes chaos, costs spiral, and the project loses control of its own scope.

Section 4 — What We Have Covered

Check your understanding against the learning outcomes

LO1 A requirement is a condition or capability that must be satisfied. The three dimensions — functional (WHAT), non-functional (HOW WELL), and domain (WHY) — cover all types.

LO2 Ambiguity is the leading cause of requirements failure. A good requirement must be clear, complete, consistent, verifiable, traceable, and feasible.

LO3 RE is the systematic process of eliciting, analysing, specifying, validating, and managing requirements — practised very differently in plan-driven and agile contexts.

LO4 Elicitation techniques include interviews, workshops, observation, prototyping, and document analysis. The golden rule: ask what people DO, not what they want.

LO5 The SRS is the formal plan-driven specification document. The product backlog is the agile alternative — both serve the same purpose through very different approaches.

LO6 Validation uses the 5C check (Correct, Complete, Consistent, Capable, Checkable). Requirements management uses change control and traceability to keep requirements under control.

LO7 AI assists in elicitation (transcription, summarisation), specification (drafting stories, rewording vague requirements), and validation (quality checking). Humans own the outcome.

Section 5 coming next — System Modelling: use case diagrams, activity diagrams, class diagrams and more.

References and Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th edition, Addison-Wesley, 2015
Chapter 4: Requirements Engineering — primary reading for this section

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th edition, McGraw-Hill, 2020
Chapters 7–8: Principles that guide practice · Requirements engineering

Requirements Engineering: From System Goals to UML Models to Software Specifications

Axel van Lamsweerde, Wiley, 2009
For deeper study — reference for goal-oriented RE and formal specification

Online Resources

- agilealliance.org — Agile glossary including user stories, acceptance criteria, and backlog refinement
- IEEE Standard 29148:2018 — Requirements Engineering standard (ISO/IEC/IEEE)
- modernrequirements.com — free RE learning resources and tool guides

Coming Up Next

Section 5: System Modelling

Use case diagrams · Activity diagrams · Class diagrams · Sequence diagrams · State machines

Tutorial this week:

Write three requirements for a student portal — one functional, one non-functional, one domain. Apply MoSCoW to a set of given requirements.